# LAB 3

AIM – Perform Requirement Modelling using Entity Relation Diagram (Structural Modelling) for your project.

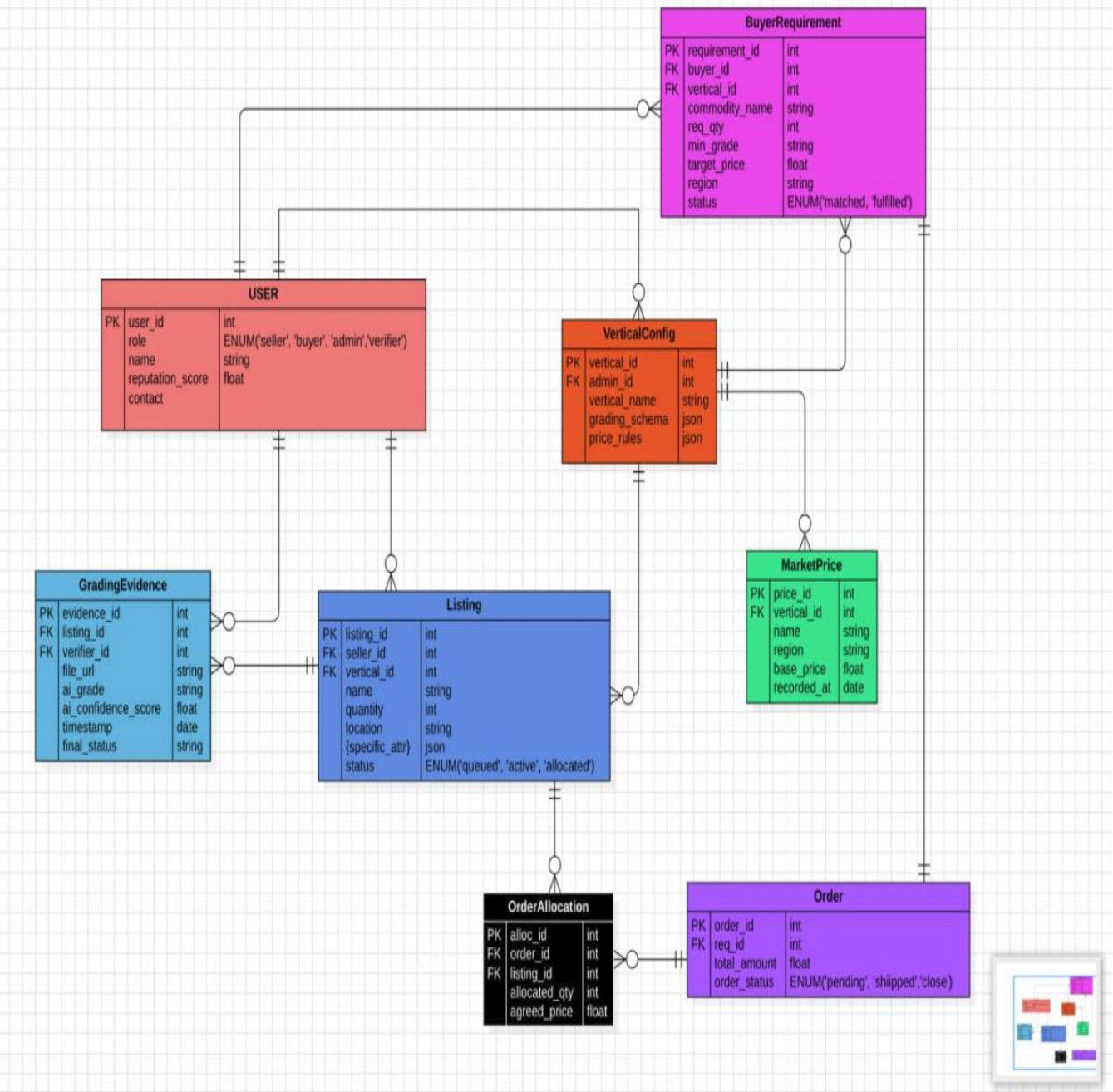

## TEAM MEMBERS -

**ARYAN RAJ (24BDS1094)**

**YASH RAJ (24BDS1209)**


## ER DIAGRAM -

(next page)




## Architectural Highlights

1. **The "Plug-in" Architecture (VerticalConfig):** Instead of creating separate tables for different commodities, the system uses a central VerticalConfig entity. Notice the

grading_schema and price_rules are stored as json. This allows an Admin to define completely different rules for textiles vs. agriculture dynamically.

2. **Dynamic Listing Attributes ({specific_attr}):** In the Listing table, the {specific_attr} is also modeled as a JSON field. This is a critical structural decision. It means a seller can upload moisture content for wheat or GSM for cotton into the same table structure, adhering to the rules defined in VerticalConfig.
3. **Algorithmic Order Fulfillment (OrderAllocation):** Unlike a standard e-commerce cart, MSME buyers often need bulk quantities that a single small seller cannot fulfill. The design uses OrderAllocation as a junction table. It maps a single Order to multiple Listing records, allowing the system's matching engine to split a large buyer requirement across multiple smaller sellers to reach the req_qty.

## Entity & Relationship Breakdown

You can use these descriptions to populate the documentation section of your lab report:

**1. Identity & Access Management**

- **USER Entity:** Acts as the central actor table. The role ENUM (seller, buyer, admin, verifier) dictates relationships across the platform. It has a 1-to-Many relationship with Listing, BuyerRequirement, GradingEvidence (as a verifier), and VerticalConfig (as an admin).

**2. Core Configuration & Market Data**

- **VerticalConfig Entity:** The configuration blueprint. It shares a 1-to-Many relationship with Listing, BuyerRequirement, and MarketPrice, ensuring that all catalog items, buyer needs, and pricing feeds are strictly categorized by their vertical.
- **MarketPrice Entity:** Stores external reference prices based on vertical_id and region, which the platform's price calculators use to suggest fair values.

**3. Cataloging & Quality Assurance**

- **Listing Entity:** Represents the seller's available commodity lot.
- **GradingEvidence Entity:** Handles the proof of quality. It has a Mandatory 1-to-Many relationship with Listing (a listing must have evidence). It tracks the ai_confidence_score and the human verifier_id to maintain a strict audit trail for dispute resolution.

**4. The Matching Engine (Requirements to Fulfillment)**

- **BuyerRequirement Entity:** Represents the buyer's posted need, including req_qty, min_grade, and target_price.
- **Order Entity:** Generated when a requirement finds a match. It tracks the total_amount and fulfillment order_status.
- **OrderAllocation Entity:** The critical associative entity (junction table) resolving the Many-to-Many relationship between Order and Listing. It records exactly how much (allocated_qty) of a specific seller's listing was used to fulfill a broader order at an agreed_price.